

Staff Engineer Technical Exercise — Data Integrations

Scenario

SyncMetrics Inc. aggregates weather data from multiple third-party APIs, normalizes it, and writes unified output files for downstream analytics import. Build an extensible ingestion pipeline in C# that fetches, transforms, and outputs normalized weather data.

Requirements

Pipeline — Open-Meteo API

Fetch 7-day forecast data from [Open-Meteo](#) (free, no key required) for a configurable set of locations. At minimum:

Location	Latitude	Longitude
New York	40.7128	-74.0060
London	51.5074	-0.1278
Tokyo	35.6762	139.6503

Endpoint:

```
GET https://api.open-meteo.com/v1/forecast?latitude={lat}&longitude={lon}

&daily=temperature_2m_max,temperature_2m_min,precipitation_sum,windspeed_10m_ma
&timezone=auto&forecast_days=7
```

Fields to ingest (all daily):

API field	Unit	Description
temperature_2m_max	°C	Daily high
temperature_2m_min	°C	Daily low
precipitation_sum	mm	Total precipitation
wind_speed_10m_max	km/h	Max wind speed
uv_index_max	index	Max UV index

Deliverables:

- Locations fetched concurrently
- Normalized tab-delimited output — design the schema yourself
- Processing summary on completion
- Runnable with `dotnet run` and passing tests with `dotnet test`

Architecture

Design the pipeline so a second API source with a completely different JSON shape can be added with minimal changes. HTTP, parsing, transformation, and output are separate

concerns. HTTP calls are behind an interface.

Error Handling

Handle HTTP failures, malformed JSON, missing fields, and unparseable values. No silent failures.

Testing

Cover: response parsing (valid and malformed), transformation logic, output formatting, and end-to-end pipeline with mocked HTTP responses.

Bonus

- Retry logic for transient HTTP failures
- Config-driven field mapping

What We're Evaluating

Area	What we're looking for
Architecture	Pipeline decomposition, interface boundaries, extensibility
Integration patterns	HTTP abstraction, non-trivial JSON parsing, schema normalization
Error handling	Resilience, structured error reporting
Testability	Mocked HTTP, meaningful edge-case coverage
AI usage	Where AI was used vs. where human judgment overrode it — document this
Staff judgment	What was generalized vs. kept simple, trade-offs articulated

AI Usage

This exercise expects AI and agentic workflows to be a core part of how you build it — not a supplement. Use Copilot, Cursor, Claude, ChatGPT, or any agentic tooling you'd reach for in production work.

Include an `AI.md` (10–15 lines) covering:

- How AI was used across the pipeline — what it drove, what it scaffolded
- One place where you overrode or corrected AI output, and why
- Any part of the solution where you deliberately chose not to use AI, and the reasoning

Submissions that show no meaningful AI usage will not advance. We're evaluating your ability to work effectively with these tools, not around them.

Submission

Return a zip or repo link. Include a short `README.md` — setup, how to run, any assumptions or trade-offs worth noting. Don't over-document; write what you'd want a new teammate to know.